

编译器设计专题实验课2026

2026.04



一、实验内容

1. 实验一共6-8个实验，不是每个都必须做，每个实验内容分必做和选做部分，必做部分一定要完成，选做根据时间和个人精力完成，必做+选做模式。
2. 设置1次验收，验收在80%部分，验收成绩占比20%
3. 考勤部分，钉钉打卡或者雨课堂签名，10%。
4. 完成每次实验，每人独立提交自己的报告。报告占比70%，实验报告独立上传思源学堂，每次设置上传截止时间。
5. 总成绩计算=平时*0.3+报告*0.7
6. 编程环境：自选



二、报告和要求

实验模式设计，便于大家按照目标，分配时间，如期完成任务验收，只需要演示基本功能，

关于报告，建议采用如下格式，去AI化

编译器设计专题实验

目录

实验X：

实验内容（必做）

实验结果：截图

实验内容（选做）

实验结果：截图

个人总结：

▶ 实验1：以云环境为例

在阿里云的PAI-DSW，

使用C/C++的web ide环境完成第一个词法分析的作业

1. 打开Terminal:

1. 在DSW页面顶部菜单栏中单击 **Terminal** 。

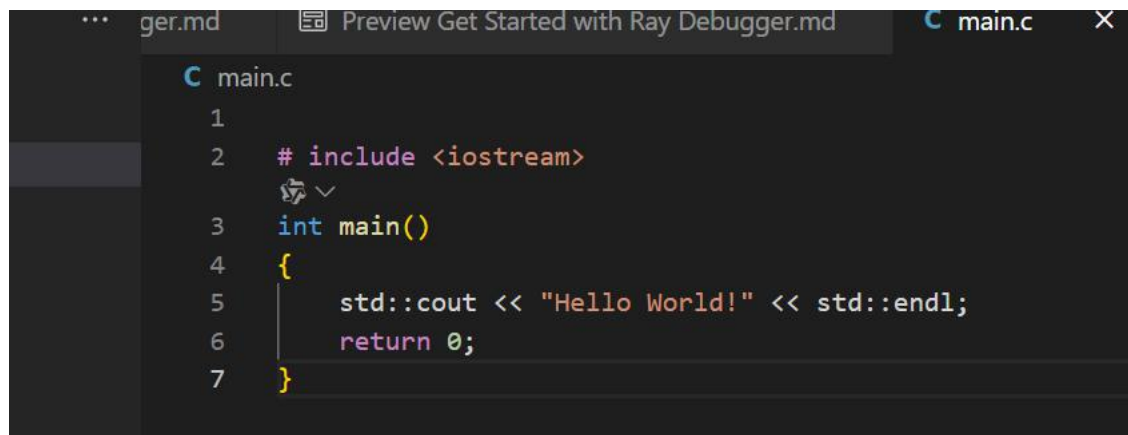
2. 更新包管理工具并安装g++

```
sudo apt update  
sudo apt install -y build-essential
```


▶ 实验1：以云环境为例

在阿里云的PAI-DSW，使用C/C++的web ide环境完成第一个词法分析的作业

1.试一下hello word。



```
ger.md Preview Get Started with Ray Debugger.md main.c X
C main.c
1
2 #include <iostream>
3 int main()
4 {
5     std::cout << "Hello World!" << std::endl;
6     return 0;
7 }
```

```
sudo apt update
sudo apt install -y build-essential
```

► 实验1：以云环境为例

在阿里云的PAI-DSW，使用C/C++的web ide环境完成第一个词法分析的作业

2. 编译执行

- 在Terminal中导航至代码目录：

```
bash  
  
cd /path/to/code
```

- 执行编译命令：

```
bash  
  
g++ -o main main.cpp
```

- 运行可执行文件：

```
bash  
  
./main
```

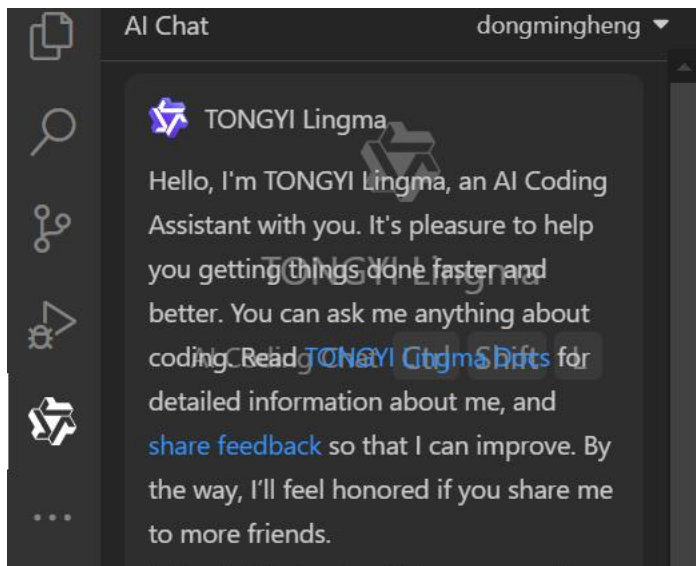


▶ 实验1：以云环境为例

hello word结果

```
compilation terminated.  
root@dsw-544537-775d865767-dcvnw:/mnt/workspace# g++ -o main main.c  
root@dsw-544537-775d865767-dcvnw:/mnt/workspace# ./main  
Hello World!  
root@dsw-544537-775d865767-dcvnw:/mnt/workspace#
```

在这个过程中，请安装通义灵码，体验tab编程



▶ 实验1

Gcc, gdb, g++, make, 已经安装

```
root@dsw-544537-77dd59c885-p9m7r:/mnt/workspace# gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@dsw-544537-77dd59c885-p9m7r:/mnt/workspace# g++ --version
g++ (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@dsw-544537-77dd59c885-p9m7r:/mnt/workspace# make --version
GNU Make 4.3
为 x86_64-pc-linux-gnu 编译
Copyright (C) 1988-2020 Free Software Foundation, Inc.
许可证: GPLv3+: GNU 通用公共许可证第 3 版或更新版本<http://gnu.org/licenses/gpl.html>。
本软件是自由软件: 您可以自由修改和重新发布它。
在法律允许的范围内没有其他保证。
root@dsw-544537-77dd59c885-p9m7r:/mnt/workspace#
```

▶ 实验1

一、实验任务

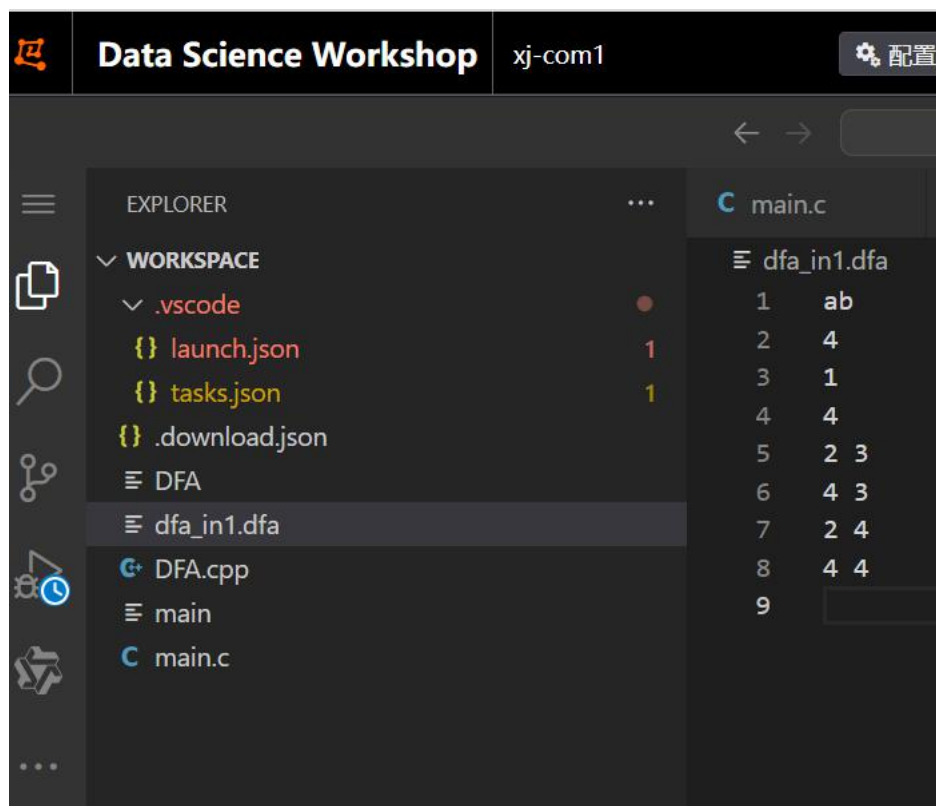
在当前环境下，编写一个C/C++语言程序，模拟DFA运行实现过程。

二、实验内容

- 1.DFA输入包括“字符集”、“状态集”、“开始状态”、“接受状态集”、“状态转换表”；
- 2.上述输入的五元组保存在一个文本文件中，
- 3.对DFA进行检查，开始状态是否唯一，包含在状态集中，接受状态集是否为空，包含在状态集中，
- 4.DFA的输出，输入待显示的字符串的最大长度N，输出以上定义的DFA的语言集中长度 $\leq N$ 的所有规则字符串；
- 5.DFA的规则判定，输入（或用字符集随机生成）一个字符串，模拟DFA识别字符串的过程判定该字符串是否是规则字符串（属于DFA的语言集）；

▶ 实验1

完成DFA，要求有输入和输出截图（必做）

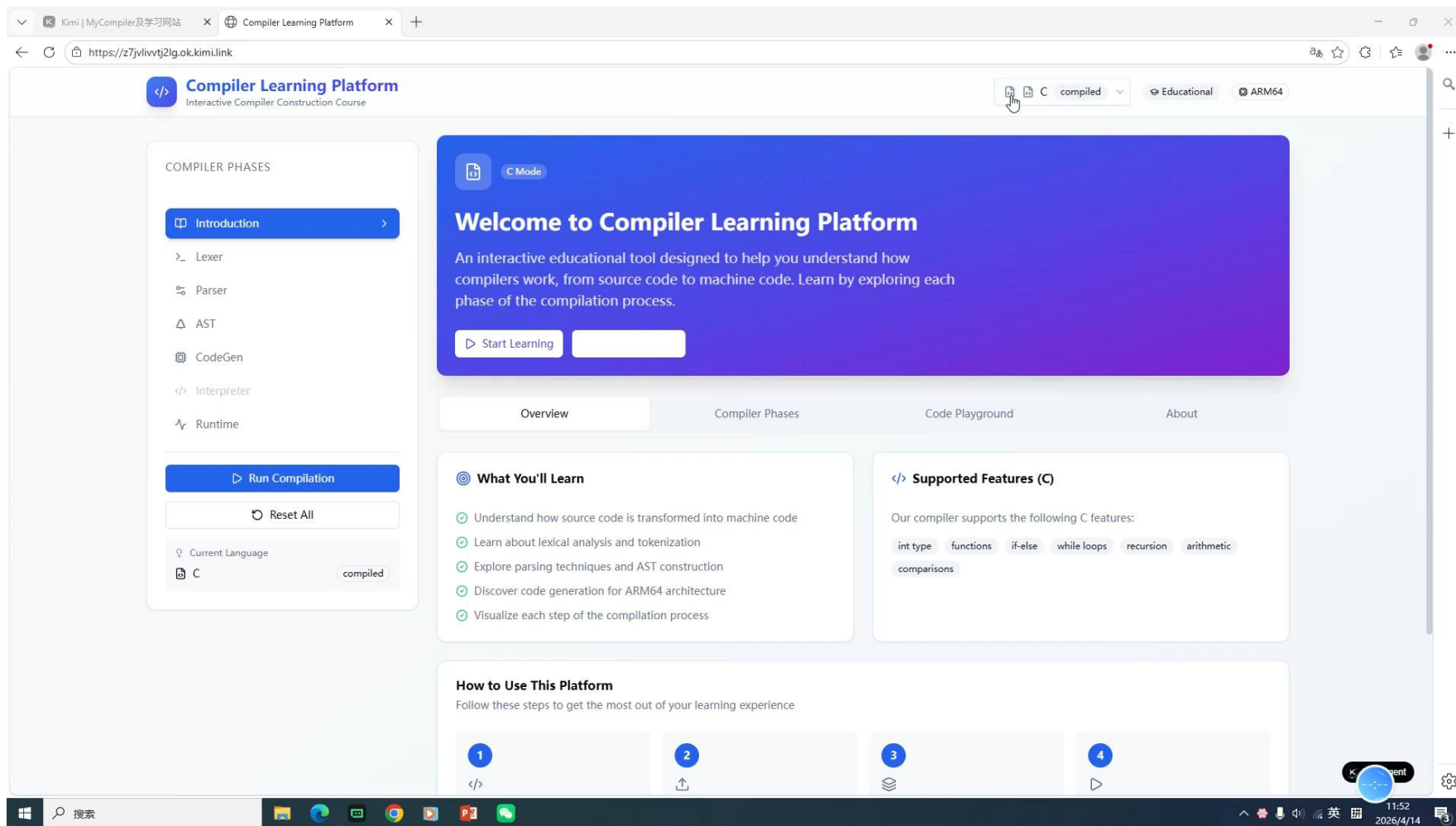


The screenshot shows the Data Science Workshop IDE interface. The Explorer panel on the left lists the workspace contents, including .vscode files (launch.json, tasks.json, .download.json), DFA, dfa_in1.dfa, DFA.cpp, main, and main.c. The main editor displays the content of main.c, which contains a DFA implementation. The DFA is defined as a table with 9 states and 4 transitions.

State	0	1	2	3
1	ab			
2	4			
3	1			
4	4			
5	2 3			
6	4 3			
7	2 4			
8	4 4			
9				

请完成教材1.6节 中心文法的DFA编程实现

实验总体架构 AI的编程新范式



▶ AI辅助编程



GitHub Copilot

无处不在的AI开发伙伴

核心功能

- 深度集成IDE，提供行内/函数级补全
- 跨文件上下文感知，理解项目逻辑
- 支持自然语言交互，解释代码与查错

适用场景

日常业务开发、快速原型验证、旧代码重构与长期维护。



Claude Code

深度思考的代码顾问

核心功能

- 超长上下文，可“阅读”整个项目代码库
- 生成高可读性代码并附带详细解释
- 强大的逻辑推理，解决复杂算法难题

适用场景

理解大型复杂系统架构、设计核心业务算法、深度代码审查。



Trae

AI原生智能开发环境

核心功能

- 对话即编程，专注业务逻辑
- 全流程自动化，端到端部署
- 深度中文优化，精准理解术语

适用场景

独立开发者/学生、中文开发团队、需要私有化部署的企业。

本实验需要完成一个能配合教材主文法的编译器，目标是ARM64，大家对于ai代码工具掌握情况怎样，预计花多少时间可以完成此项实验

-  **A 非常熟悉，1天以内**
-  **B 一般熟悉，2，3天**
-  **C 不太熟悉，需要1周**
-  **D 完全不熟悉，更长时间**